

From Vibe Coding to Spec-Driven Agentic Engineering

A Hands-On Implementation using Claude Code

PART XII

Instructor: Muhammad Arif Butt, PhD

About The Instructor



Dr. Muhammad Arif Butt
Asst. Prof. Department of Data Science
University of Punjab, Lahore
arif@pucit.edu.pk
https://arif.excaliat@gmail.com
https://arifbutt.me
https://www.linkedin.com/in/dr-arif-butt/
https://youtube.com/learnwitharif
https://github.com/arifpucit
https://huggingface.co/arif-butt

- **Education:**
  - Graduation from Pakistan Military Academy, Kakul
  - MPhil CS from University of Punjab, Lahore
  - PhD CS from University of Punjab, Lahore
- **Experience:**
  - Served in field/staff/instructional posts in Pakistan Army
  - Assistant Professor, Department of Data Science
  - Founder Excaliat (https://excaliat.com/en)
  - Founder FalconHunt (https://falconhunt.org/)
  - Co-Founder Tbox Solutionz (https://tboxsolutionz.com/)
- **Research Interest:**
  - Embedded and Real-Time Operating Systems
  - Vulnerability Analysis, Binary Exploitation & Exploit Development
  - AI-Driven Cybersecurity and Securing AI systems

Scheme of Sessions

Part - I:	Overview of Agentic Coding Assistants
Part – II:	Claude Ecosystem, Installing & using Claude Code
Part – III:	Slash Commands, Context Window & `.claude` Directory
Part – IV:	Workflow of Building a Basic Vulnerable Web Application
Part – V:	Exploiting Vulnerable Web Application
Part – VI:	Workflow of Implementing Features and Fixing Bugs
Part – VII:	Collaborative Workflow: Claude Code + GitHub + Jira
Part – VIII:	Built-in Skills & Vuln-Web-App Current Status
Part – IX:	Creating & using Custom Skills & Skill Marketplace
Part – X:	Deep Dive: Conceptual Understanding of MCP
Part – XI:	MCP in Action
Part – XII:	**Using / Creating Built-in & Custom Subagents**
Part – XIII:	Background Agents, Git Worktrees, & Agent Teams

Recap: Six Pillars of the Claude Code Ecosystem

Knowledge — What Claude knows

CLAUDE.md

ALWAYS LOADED

Persistent project context loaded at the start of every session. Files merge hierarchically — user, project, subdirectory — with more specific files winning.

Skills

USER OR CLAUDE INVOKED

Reusable instructions plus bundled scripts and assets, loaded only when needed via progressive disclosure. Run manually with /skill-name or let Claude trigger one from its description. Absorbed custom slash commands.

Execution — How work gets done

MCP servers

MODEL INVOKED

Brokered access to external systems — APIs, databases, and developer tools — exposed to Claude as callable tools over a standard protocol.

Subagents

DELEGATED

Specialized workers that run in their own isolated context window and return only a summary — heavy research and parallel tasks never pollute the main thread.

Operations — Enforcing and sharing

Hooks

EVENT TRIGGERED

Deterministic code that fires on lifecycle events — formatting, logging, validation, notifications. Enforce rules with code, not prompts.

Plugins

DISTRIBUTION UNIT

Bundles skills, subagents, hooks, MCP configs, and LSP servers* into one installable, versioned package — the canonical way to share a setup across teams and marketplaces.

* LSP servers give Claude IDE-grade code intelligence — go-to-definition, find references, live diagnostics — via the Language Server Protocol. Configured through plugins; the language server binary is installed separately.

Why Agents?

Why Agents?

A single AI session hits four walls. This series builds a fix for each and we start with the first.

01

START HERE

Context pollution

one window fills up with noise

→ **Subagents**

give each task its own context

02

Blocking / waiting

you sit idle while one task runs

→ **Background agents**

run work off the main thread

03

File collisions

agents edit the same files at once

→ **Git worktrees**

give each its own workspace

04

No coordination

agents can't talk to each other

→ **Agent teams**

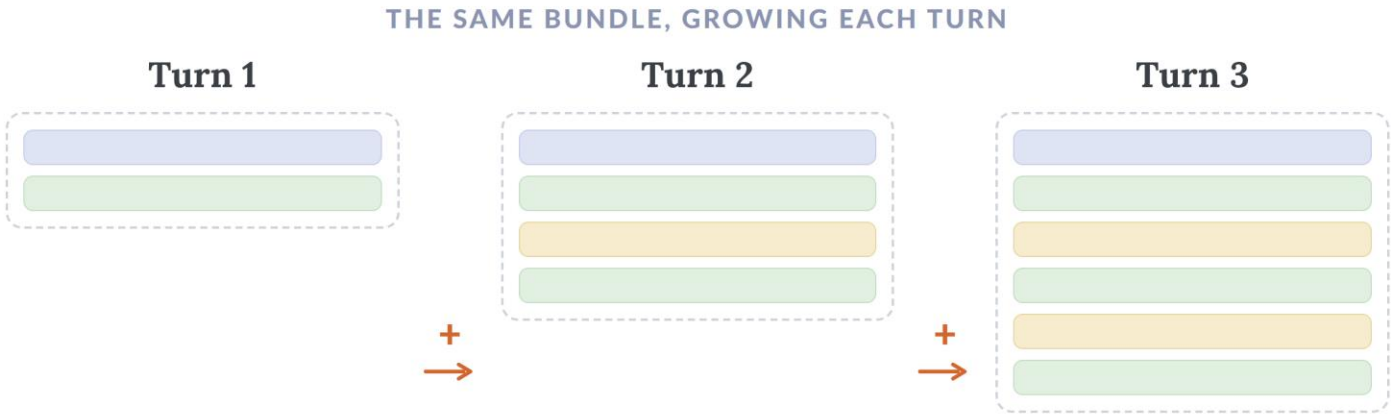
let sessions message each other

Context Window

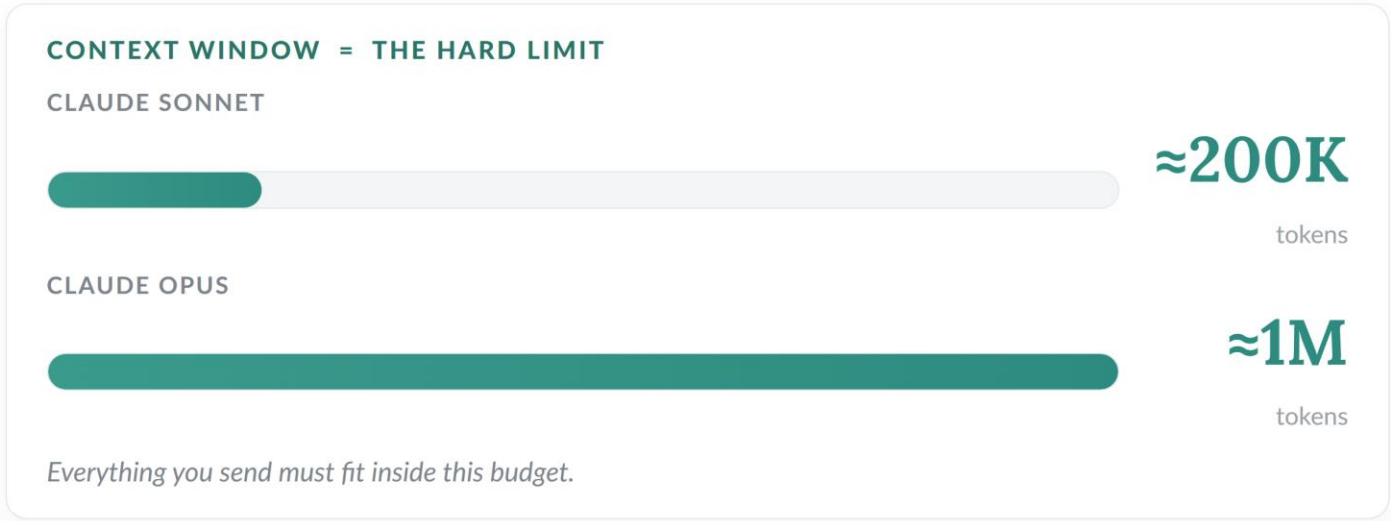
The model has no memory — the app re-sends everything

An LLM is stateless. To continue a conversation, your app re-sends the entire history with every single API call.

- **Stateless by design**
  Each API call stands alone — nothing carries over from earlier turns on its own.



- **History re-sent every turn**
  Every prior prompt, reply, file read and tool output is bundled and sent again as one input.



- **It all must fit one budget**
  That bundle is measured in tokens and must fit inside a fixed limit — the context window.

Chatbot vs Coding Agent

Same trick, very different weight

Both re-send their whole history each turn — but what sits *inside* that history is worlds apart.

CHATBOT — LIGHT

- System prompt
- User message — short text
- Assistant reply — short text
- … a few more short turns

Grows slowly · stays light · rarely nears the limit

A 50-message chat is still only a few thousand tokens.

CODING AGENT — HEAVY

- System prompt
- CLAUDE.md
- Tool definitions
- Your prompt
- **Entire codebase read in**
- Tool outputs & logs
- Model reasoning

Grows fast · gets heavy · fills the window

One review can pull in 40+ files — tens of thousands of tokens.

A chat history is light text. A codebase is huge — and now it rides along on **every single turn**.

Solution: Sub-agents

Every turn drags the whole codebase along — as context fills up, two things break, and a sub-agent is the cleaner fix.

WHEN CONTEXT GETS HEAVY, TWO THINGS BREAK

CONTEXT WINDOW OVERFLOW

The bundle grows past the token limit — older content gets truncated, or the call fails outright.

LOST IN THE MIDDLE

Even when it fits, the model attends best to the start and end. Detail buried in the middle gets missed, so answers degrade.

MANUAL FIXES — YOU'RE MANAGING THE WINDOW BY HAND

`/context`	see what's filling it	`/compact`	summarize to shrink it
`/clear`	wipe and restart	`/resume`	reopen a past session

THE FIX — HAND NOISY TASKS TO A SUB-AGENT

The codebase is read in a separate window — only a short summary returns to your main session.

WITHOUT A SUBAGENT

15,000

tokens kept in the main context

≈ 92%

less load on your main context

WITH A SUBAGENT

1,200

tokens returned to main context

1 Delegate the noisy job

you hand off the audit

2 Runs in its own window

the 40 files load there, not in your session

3 You get back a summary

only the findings return to you

Same work. A fraction of the main-context cost.

What is a Sub-agent?

What Is a Subagent?

A subagent— is an AI assistant that run in its own isolated context window, doing all the heavy lifting and handing back only the result.

MAIN SESSION
the agent you talk to

Stays lean & in control.

YOUR CONTEXT — mostly free

delegates one task

returns a summary

SUBAGENT
spawned for one task

Its OWN context window.

ITS CONTEXT — the messy work lives here

Advantages of using subagents

- Context isolation
- Specialization
- Modularity
- Parallelism

Top Use Cases

- Codebase Exploration
- Independent code review
- Security audit
- Parallel independent tasks
- Multi-stage pipelines

Built-in vs Custom Subagents

Some subagents come with Claude Code out of the box — you don't create or configure them; Claude just uses them when they help.

BUILT-IN
ships with Claude Code

- Zero setup — already there
- Claude invokes them automatically
- You rarely call them by name
- Tuned for the most common jobs

CUSTOM
you build these yourself

NEXT UP

- You write the .md file
- You choose its tools & model
- Your own named specialist
- Covered in the next section

This section is about the built-in ones — the specialists already working behind the scenes, before you write a single file.

Does the Main Agent Delegate Everything?

No — the main agent does most work itself. It only calls in a built-in when a job would flood its own context.

IT DOES IT ITSELF
direct — no subagent

- `Read one file`
- `Make one edit`
- `Run one command`

small, quick work stays in the main context

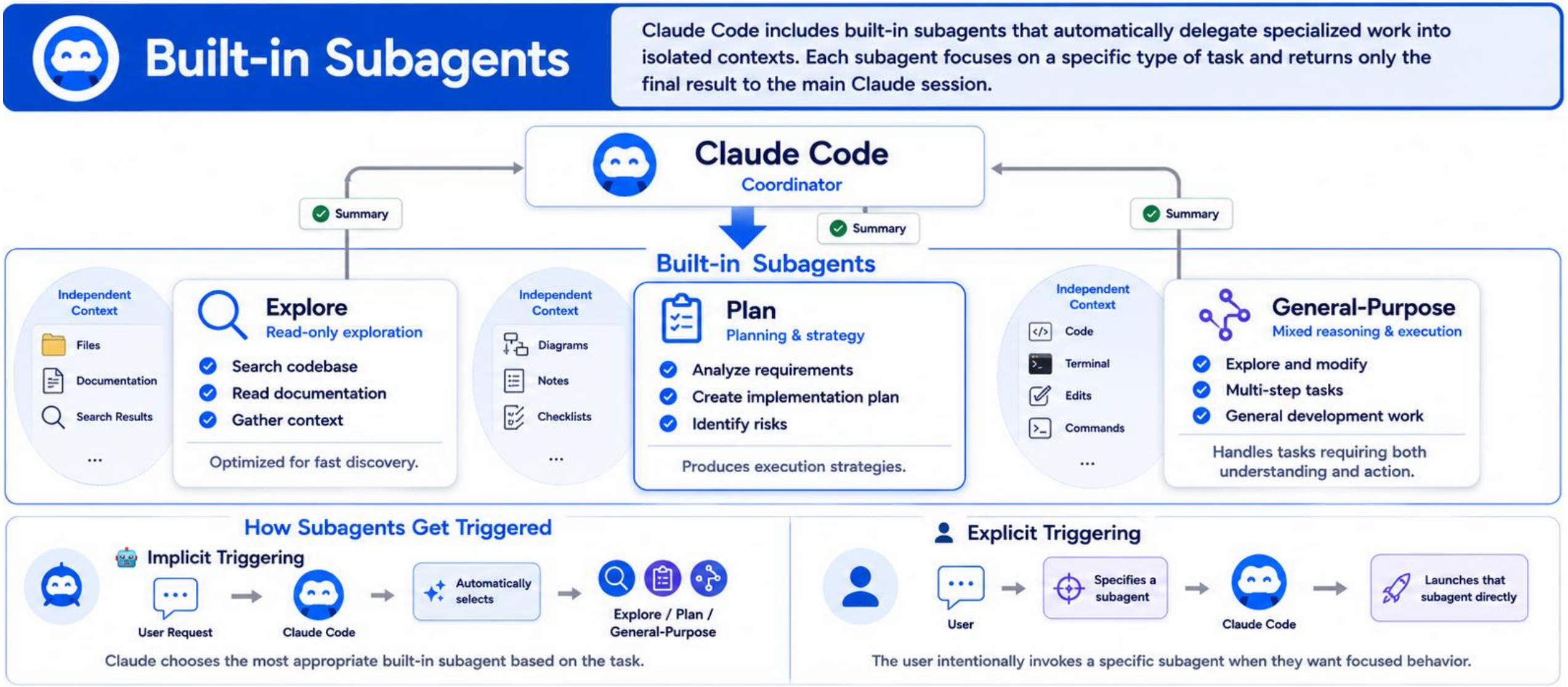
IT DELEGATES
spins up a built-in

- `Search the whole codebase`
- `Map how input reaches the DB`
- `Explore across dozens of files`

large, noisy work runs in a subagent's window

Rule of thumb: small & direct → the main agent handles it. Large & noisy exploration → it's delegated to a built-in.

Built-in Subagents



Hands on Practice: Built-in Subagents



Custom Subagents

Custom Sub-agents in Claude Code

1 A subagent is a Markdown file on disk — where it lives & what's inside

PROJECT · .CLAUDE/ LAYOUT

```
<project-directory>/
└ .claude/
   ├ agents/            # one .md each
   │  ├ security-reviewer.md
   │  ├ dependency-scanner.md
   │  ├ patch-writer.md
   │  └ test-runner.md
   └ agent-memory/      # auto-created
      └ security-reviewer/
         ├ MEMORY.md
         └ triaged-cves.md
```

💡 Memory sits **beside** `agents/`, not inside — keyed by the agent's **name**. `agents/` holds only the **.md** files.

FRONTMATTER · THE CONFIG

```
---
name: security-reviewer
description: Reviews diffs for SQLi,
  XSS, CSRF & auth. Use proactively.
tools: Read, Grep, Glob
model: sonnet
skills: [secure-coding, owasp-top-10]
mcpServers: [github, semgrep]
permissionMode: default
memory: project
background: false
isolation: strict
---
```

BODY · THE SYSTEM PROMPT

```
You are a security reviewer. When invoked,
scan the diff and return a severity-ranked
report.
```

Frontmatter — REQUIRED

The settings block between the two `---` lines. Declares **what the agent is** and **what it may touch** — only `name` and `description` are mandatory; the rest tune it.

keys: `name` `description` `tools` `model` `memory` `skills` `+ more`

Body — REQUIRED

Everything below the second `---` is the **system prompt** — the agent's role, its job, and how to report back. Write it like any focused prompt.

covers: `role` `when-invoked` `output format`

Where a subagent can come from

↓ higher on the list wins a name clash

1	**Session**	`claude --agents`	defined at runtime for this run only
2	**Managed**	`managed .claude/agents/`	pushed by your org admins — org-wide
3	**Project**	`.claude/agents/`	committed to the repo, shared with the team
4	**User**	`~/.claude/agents/`	personal, available in every project
5	**Plugin**	`plugin:name`	ships with an installed plugin

Plus built-ins: *Explore, Plan & general-purpose ship with Claude Code and are always on.*

Same idea as a Skill file — frontmatter on top, instructions below — but a subagent runs as its **own agent, in its own context window**, and reports a result back to the main session.

Custom Sub-agents in Claude Code

2 Getting a subagent to run — the description decides, three commands invoke

◎ THE DESCRIPTION FIELD = THE TRIGGER

```
# × too vague — Claude can't tell when it applies
description: reviews code

# ✓ specific & proactive — names the situations
description: Use proactively to review diffs
  for SQLi, XSS, CSRF & auth flaws.
```

✓ Claude reads only **name + description** when deciding whether to delegate — so write it like a **routing rule**.

>_ THREE WAYS TO INVOKE IT

```
> use the security-reviewer on login.php  # Claude decides
> @security-reviewer review login.php        # names it
> claude --agent security-reviewer        # whole run
```

The description is the trigger — MOST IMPORTANT

Claude matches your request against this one line to decide whether to use the agent **at all**. Name the exact phrases and situations that should fire it, and add **"use proactively"** so Claude reaches for it on its own.

Natural language — FLEXIBLE

`use the security-reviewer on login.php` — Claude decides if it fits the description. The everyday path.

@mention — RELIABLE

`@security-reviewer review login.php` — names the agent directly, so it's **guaranteed to run**.

Session-wide — WHOLE RUN

`claude --agent security-reviewer` — the entire session runs **as that agent** from the start.

Start with natural language. Reach for **@mention** when you have several similar agents and want to be sure the right one runs.

Custom Sub-agents in Claude Code

3 Tools are an agent's permissions — the actions it's allowed to take

OUR SECURITY-REVIEWER'S TOOLS

```
tools: Read, Grep, Glob    # an allow-list

✓ Read          look at files
✓ Grep / Glob   search the code
× Edit / Write  not granted — can't change code
× Bash          not granted — can't run commands
```

Omit `tools` and the agent inherits **every** tool. **List** them and it can use **only** those — fewer tools means a smaller blast radius.

In practice tool lists fall into **three tiers** — pick the smallest one that still lets the agent do its job:

READ-ONLY

```
tools: Read, Grep, Glob
```

"look, don't touch"

`security-reviewer` · `dependency-auditor`

BALANCED

```
tools: Read, Edit, Bash, Grep, Glob
```

"build, don't break"

`patch-writer`

FULL ACCESS

```
(omit tools → inherits everything)
```

"trusted tasks only"

use sparingly

Restrict on purpose. A reviewer with no `Edit` tool literally **cannot** alter the code it reviews — the capability simply isn't there to misuse. Like onboarding an intern: you don't hand over the whole production database on day one.

The rule: give each agent exactly the tools its job needs — no more. That's your main safety lever.

Custom Sub-agents in Claude Code

4 Agent memory — how a subagent remembers — what it is, why it helps & how it's created

① TRIGGER · ONE LINE OF FRONTMATTER

```
---
name: security-reviewer
memory: project     # turns it on
---
```

AT RUNTIME · ON EVERY INVOCATION

- First **200 lines** of `MEMORY.md` are injected into the agent's system prompt
- `Read` · `Write` · `Edit` are auto-enabled so it can **curate its own notes**
- New insights are **appended after each run** — memory compounds over time
- Notes are treated as **hints** — verified against the real code before acting

💡 You never mkdir anything. Set the **scope** (`user` / `project` / `local`) and the folder + `MEMORY.md` are **created for you** on first run.

② RESULT · AUTO-CREATED ON DISK

```
.claude/
└─ agent-memory/              # per agent
   ├─ security-reviewer/      # keyed by name
   │  ├─ MEMORY.md            # loaded each run
   │  └─ triaged-cves.md      # agent's notes
   └─ dependency-scanner/     # separate silo
      └─ MEMORY.md            # invisible
```

INSIDE security-reviewer/MEMORY.md

```
## Recurring findings
- Unsanitized query in db.js → SQLi
- XSS via unescaped template in ui.tsx
- Team rule: always parametrize queries
```

🔒 **Siloed by name.** Each agent sees only its own memory — the reviewer can't read the scanner's notes, and vice-versa.

What agent memory is

NEW IN v2.1.33

A **per-agent Markdown knowledge store that persists across invocations**. Flip on the `memory:` field and the subagent can learn, remember, and build expertise — instead of starting from a blank slate every single run.

Why it's used — from amnesia to expertise

✕ WITHOUT MEMORY	✓ WITH MEMORY
✕ Wakes up blank on every run	✓ Recalls patterns, decisions & fixes
✕ You re-explain the same conventions	✓ Gets sharper the more it runs
✕ Re-flags patterns you already rejected	✓ Stops repeating rejected suggestions
✕ Burns tokens & focus repeating context	✓ Less context to re-feed each time

How it's created — pick a scope

The scope you set on **memory:** decides **where** the folder lives and **who** shares it. Folder + **MEMORY.md** are auto-created on first run.

`user`	`~/.claude/agent-memory/<name>/`	**Cross-project**, not shared — the recommended default
`project`	`.claude/agent-memory/<name>/`	**Committed to the repo** — shared with the whole team
`local`	`.claude/agent-memory/<name>/`	**Personal** to this project — git-ignored, just for you

Frontmatter grants the memory; `agent-memory/` **is where it lives** — auto-created, keyed by name, one silo per agent, and injected fresh into the prompt on every run.

Five Ways to Create a Custom Subagent

Five ways to create a custom subagent

◀ more control · more effort

faster · less writing ▶

1 — 2 — 3 — 4

1 Write the file yourself

HOW IT WORKS

Create `.claude/agents/` and a **name.md** file by hand — YAML frontmatter plus the system prompt. Claude Code picks it up within seconds.

EXAMPLE

```
# .claude/agents/security-reviewer.md
---
name: security-reviewer
description: Flags SQLi/XSS.
  Use proactively.
tools: Read, Grep, Glob
model: sonnet
---
You are a security reviewer…
```

PROS

- ✔ **Full control** of every field & the prompt
- ✔ No tooling — just a text file
- ✔ Deepest grasp of the format

WATCH-OUTS

- ! Slowest of the four
- ! Must know the frontmatter fields

BEST FOR
bespoke, precise agents

Control ●●●●
Speed ●○○○

2 Use the `--agents` flag

HOW IT WORKS

Pass **JSON** when you launch Claude Code. Same fields as the file (`prompt` is the body). The agent lives only for that session — never written to disk.

EXAMPLE

```
claude --agents '{
  "code-reviewer": {
    "description": "Reviewer…",
    "prompt": "Senior review…",
    "tools": ["Read","Grep"],
    "model": "sonnet"
  }
}'
```

PROS

- ✔ No file needed — great for **CI & scripts**
- ✔ Define several agents at once
- ✔ Perfect for quick throwaway tests

WATCH-OUTS

- ! **Ephemeral** — gone when the session ends
- ! Hand-writing JSON is fiddly

BEST FOR
quick tests & automation

Control ●●●○
Speed ●●○○

3 Ask Claude to write it

HOW IT WORKS

Describe the agent you want and where to save it — Claude writes the **.md** (name, description, tools, model, prompt), then you review. The **recommended** quickstart.

EXAMPLE

```
You › "Create a read-only
  security-reviewer in
  .claude/agents/ that flags
  SQLi & auth bugs. Use Sonnet."

Claude writes → security-reviewer.md
  name · description · tools ·
  model · system prompt
```

PROS

- ✔ **Fastest to a working file**
- ✔ Great for beginners
- ✔ Claude fills in frontmatter & prompt

WATCH-OUTS

- ! Review the **description & tools** for fit
- ! Less deliberate control of details

BEST FOR
quick starts & everyday agents

Control ●●○○
Speed ●●●○

4 Install from a plugin

HOW IT WORKS

Add a plugin from a **marketplace** with `/plugin` . Its agents appear under a scoped name — nothing to write, you're using one someone else built.

EXAMPLE

```
/plugin  → marketplace → install

# or from your shell:
claude plugin marketplace add
  anthropics/claude-plugins
claude plugin install <name>

→ @agent-<plugin>:code-reviewer
```

PROS

- ✔ **Fastest** — no writing at all
- ✔ Ready-made & maintained
- ✔ Shareable across a team

WATCH-OUTS

- ! You inherit someone else's prompt & tools — **audit first**
- ! Plugin agents ignore `hooks` / `permissionMode`

BEST FOR
common, already-solved tasks

Control ●○○○
Speed ●●●●

5 **A fifth route — the `/agents` command (interactive).** Run `/agents` inside Claude Code to create, edit & delete your **own** subagents: *Create new → pick Project or Personal scope → Generate with Claude, or fill it in.* You own the resulting `.md` and edit it to change the agent; the name stays short (e.g. `researcher`).

Version note: the in-terminal creation wizard ships on Claude Code v2.1.197 and earlier; v2.1.198+ removed it and points you to ① / ③ (edit the file or ask Claude). Org admins can also deploy **managed** subagents fleet-wide, at the highest priority.

Building a Custom Subagent

Building a custom subagent: `mail-drafter`

Drafts in your voice · learns your style as you edit · **sends via Google Workspace**

◀ SET UP ONCE — RUNS & LEARNS EVERY TIME ▶

1 — 2 — 3 — 4

1 Write the prompt

HOW IT WORKS

Tell Claude Code what agent you want and where to save it. It writes the **.md** file — frontmatter plus system prompt — in seconds.

THE PROMPT YOU GIVE

```
You › Create a subagent at
  .claude/agents/mail-drafter.md

It drafts emails in my voice, shows
each draft as plain text for my
approval, and only sends after
I confirm.

Tools: Read + the google-
workspace draft/send tools.
Model: sonnet. Give it project
memory in ./.claude/ so it
learns my style over time.
```

→ Claude Code writes `mail-drafter.md`

2 The generated file

HOW IT WORKS

YAML frontmatter sets the agent's **identity, tools, model & memory**; the body is its system prompt.

.CLAUDE/AGENTS/MAIL-DRAFTER.MD

```
# frontmatter — the agent's config
---
name: mail-drafter
description: Use proactively to
  compose or send email. Drafts,
  waits for approval, learns style.
tools: Read,
  …google-workspace…draft_gmail,
  …google-workspace…send_gmail
model: sonnet
memory: project
---
# body — the system prompt
You are an email-drafting assistant…
## Core workflow
## Learning the user's style
## Where memory lives
```

3 First run creates memory

HOW IT WORKS

The first time it saves style rules, the agent creates a memory folder under `.claude/agent-memory/`. It reads these before every draft and appends as it learns.

WHAT APPEARS ON DISK — THREE FILES

```
project-root/
└ .claude/
   └ agent-memory/
      └ mail-drafter/
         ├ MEMORY.md              ← new
         ├ style-format-tone.md   ← new
         └ style-greetings.md     ← new
```

WHAT EACH FILE HOLDS · ILLUSTRATIVE

`MEMORY.md` — index of what it knows
`style-format-tone.md` — length, formality, phrasing
`style-greetings.md` — greetings & sign-offs

4 A sample run

YOU TYPE

```
@mail-drafter I am Jamil, please
write an email to my professor
Dr. Arif Butt asking for the
lecture notes
```

IT DRAFTS — PLAIN TEXT, FOR APPROVAL

MAIL-DRAFTER

Subject: Request for lecture notes
Assalam o Alaikum Dr. Butt,
Could you please share this week's lecture notes? Regards, Jamil

Review this before I send it?

YOU APPROVE → IT SENDS

You › **send it**

✓ Calls `send_gmail_message` via the **google-workspace** MCP server — **email sent to Dr. Arif Butt.**

…AND IT LEARNS AS YOU GO

Edit the sign-off → agent replies: **"Noted — I'll use that from now on."**
✓ saved to memory

The loop, every email: Draft in chat → You review & edit → Explicit confirm `"send it"` → `send_gmail_message`

How it learns from your edits: a `STYLE` edit (would fit any email) is saved to memory · a `CONTENT` edit (only this email) is applied once, never saved · **unsure? it asks.**

Hands on Practice: Custom Subagents



Skills vs Subagents

Instructor: Muhammad Arif Butt, PhD

Skills vs Subagents

Same file shape, different rulebook. Put the wrong field in the wrong file and it is silently ignored — not an error.

SKILL

Location
.claude/skills/name/SKILL.md

Tool control
allowed-tools — pre-approves only

Memory
none — no such field exists

Auto-routing
only when invoked, by /name

Extra files
yes — scripts, assets, references

SUBAGENT

Location
.claude/agents/name.md

Tool control
tools — a hard allowlist

Memory
memory: field, persists

Auto-routing
checked against every message

Extra files
no — a single file only

The trap: copy one file's frontmatter into the other's folder and the mismatched fields do not error — they are just dead text.

When To Use Which?

One question decides it: does it need restricted permissions, or does it need to remember things across sessions?

USE A SKILL WHEN

Reusable instructions
same checklist, run it every time

Fine running inline
no noisy output to isolate

No permission limits needed
does not touch anything sensitive

No memory needed
a fresh start every time is fine

USE AN AGENT WHEN

Needs restricted tools
must not touch certain things

Needs memory
must remember across sessions

Needs standing auto-routing
should fire without being asked

Noisy work to isolate
keep raw output out of main chat

Still unsure? ask about the specific idea instead of the rule — it is faster than re-deriving the whole decision tree.

Composing Skills & Agents

They connect through exactly two fields: one lets a skill borrow an agent, the other lets an agent borrow a skill.

SKILL → AGENT (FORK)

Field
context: fork + agent: name

What happens
the skill's task runs as that agent

Example
/full-audit forks into security-tracker

Gains
isolation for one run — not memory

AGENT → SKILL (PRELOAD)

Field
skills: [name] in agent frontmatter

What happens
full skill content loads at startup

Example
xss-hunter preloads owasp-top-10

Gains
agent follows exact team conventions

Important: fork borrows isolation for one run — it never gives the skill memory or a standing, always-on identity.

Q/A



Thank you!